2 parcial

Universidad: Politécnica de Atlautla

Asignatura: Fundamentos de Inteligencia Artificial

Programa: Ing. en Tecnologías de la Información e Innovación Digital

Tema: Actividad 2.4 Algoritmos de búsqueda ciega (BFS/DFS)

Docente: Mtro. Javier García Lara

Alumno: Anahi Fuentes Avendaño

Enlace de GitHub:

https://github.com/fana60203/FIA_G171_Anahi_Fuentes.

Grupo: 171

Fecha de entrega: 25 – Octubre - 2026

Tema: Actividad. 2.4. Algoritmos de búsqueda ciega (BFS/DFS)

Código + Comparación. Implementar BFS y DFS en Python para resolver un laberinto o puzle. Documentar: representación del grafo, complejidad temporal/espacial, camino solución, comparación (tiempo, nodos expandidos) y conclusión sobre idoneidad según problema.

Datos del problema

Dato	Valor
Tipo de problemas	Laberinto en cuadricula, un agente, un solo objetivo
Tamaño de la cuadricula	21x21 celdas (pared = 1, libre = 0)
Generación	Backtracker iterativo (semilla = 7) + apertura del 15% de paredes internas para crear ciclos.
Estado inicial S	(1, 1) (fila, columna)
Estado meta G	(19, 19)
Movimientos permitidos	Arriba, Derecha, Abajo, Izquierda (4-conectividad, sin diagonales)
Costo de cada movimiento	1 (costo uniforme)
Nodos del grafo |V|	217 celdas libres
Aristas |E|	238 pares de celdas libres adyacentes
Ciclos independientes	22 (|E| – |V| + 1): existen rutas alternativas



Planteamiento del problema.

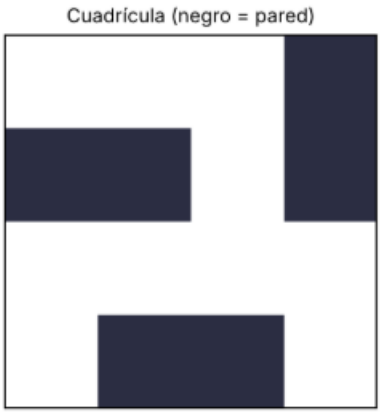
Un agente se encuentra en la esquina superior izquierda de un laberinto y debe llegar a la esquina inferior derecha moviéndose una celda a la vez en las cuatro direcciones cardinales, sin atravesar paredes. Se busca una secuencia de acciones que lleve del estado inicial a la meta y comparar cómo se comportan BFS y DFS en términos de tiempo de ejecución, nodos expandidos, longitud (costo) del camino encontrado y memoria (tamaño máximo de la frontera). Como el laberinto contiene ciclos, hay varias rutas validas, lo que permite observar la diferencia entre encontrar una solución y encontrar la mejor solución.

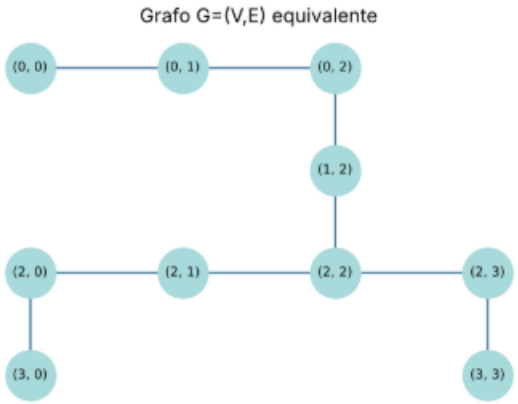
Formulación como problema de búsqueda

Componente	Definición
Estado	
Estado inicial	S0 = (1,1)
Acciones ACCIONES (s)	(Arriba, derecha, abajo, izquierda), aplicables si la celda destino es libre y esta dentro de la cuadricula
Modelo de transición	Desplaza (fila, columna) en (-1, 0), (0, +1), (+1,0) o (0,-1)
Prueba de meta	S = (19, 19)
Costo de camino	1 por accion, el costo de un camino es su número de pasos

Representación del grafo

El espacio de estados se modela como un grafo no dirigido G = (V, E): cada celda libre es un vértice y existe una arista entre dos celdas libres adyacentes (peso 1). Se almacena como lista de adyacencia mediante un diccionario de Python {(fila, col): [vecinos]}. Se eligió la lista y no la matriz de adyacencia porque el grafo es disperso: cada nodo tiene a lo más 4 vecinos, por lo que la lista ocupa O (|V| + |E|) frente a O(|V|2) de la matriz (aquí 2172 = 47,089 entradas contra 693 elementos). Además, se construye el mismo grafo con NetworkX (NetworkX developers, s. f.) para validar |V|, |E|, la conectividad y la longitud del camino optimo con nx.shortest_path_length.





Procedimiento

Ambos algoritmos comparten el esqueleto de *búsqueda en grafo*: una **frontera** de nodos por expandir, un registro de nodos alcanzados (evita ciclos) y un diccionario de **padres** para reconstruir el camino. Lo único que cambia es la estructura de la frontera (UC Berkeley CS 188, s. f.). La cola se implementa con collections.deque (Python Software Foundation, s. f.):

	BFS	DFS
Frontera	Cola FIFO (colletions.deque)	Pila LIFO (lista de python)
Siguiente nodo	El más antiguo (menor profundidad)	El más reciente (mayor profundidad)
Marca de visitado	Al generar el nodo (se inserta en padre)	Al expandir el nodo (visitado
Orden de vecinos	Arriba, derecha, abajo, izquierda)	Se apilan en reversa para explorar primero Arriba.

```
BFS (grafo, inicio, meta):
frontera ← COLA[inicio]
padre[inicio] ← NULO
mientras frontera no vacía:
n ← frontera.sacar_primero()
expandidos ← expandidos + 1
si n = meta: devolver camino(n)
expandidos++
```

```
DFS(grafo, inicio, meta):
frontera ← PILA[inicio]
padre[inicio] ← NULO; visitado ← ∅
 mientras frontera no vacía:
   n ← frontera.sacar_último()
   si n ∈ visitado: continuar
    visitado ← visitado ∪ {n};
```

para cada v en vecinos(n):
si v ∉ padre:
padre[v] ← n
frontera.agregar(v)
devolver FALLO

si n = meta: devolver camino(n)
para cada v en vecinos(n) (reversa):
si v ∉ visitado:
padre[v] ← n; frontera.agregar(v)
devolver FALLO

Medición: cada algoritmo se ejecuta 200 veces sobre el mismo grafo y se reporta la mediana del tiempo con time.perf_counter() (reduce el ruido del sistema operativo). Nodos expandidos es el número de nodos extraídos de la frontera y procesados, incluyendo el nodo meta.

Resultados

- Salida de ejecución

Laberinto 21x21 | V=217 | E=238 | ciclos=22
Inicio= (1, 1) Meta= (19, 19) conexo=True
BFS: pasos=40 expandidos=216 frontera_max=14 tiempo=0.0546 ms
DFS: pasos=62 expandidos=70 frontera_max=14 tiempo=0.0241 ms

- Camino solución

BFS (40 pasos, óptimo): Abajo×2, Derecha×4, Abajo×2, Derecha×4, Abajo×2, Derecha×6, Abajo×2, Derecha, Abajo×2, Derecha×3, Abajo×6, Izquierda×2, Abajo×2, Derecha×2.
DFS (62 pasos): Abajo×2, Derecha×4, Abajo×2, Derecha×2, Arriba×4, Derecha×6, Abajo×4, Derecha×2,
Arriba×4, Derecha×4, Abajo×6, Izquierda×2, Abajo×2, Izquierda, Abajo×2, Derecha×3, Abajo×6, Izquierda×2, Abajo×2, Derecha×2.

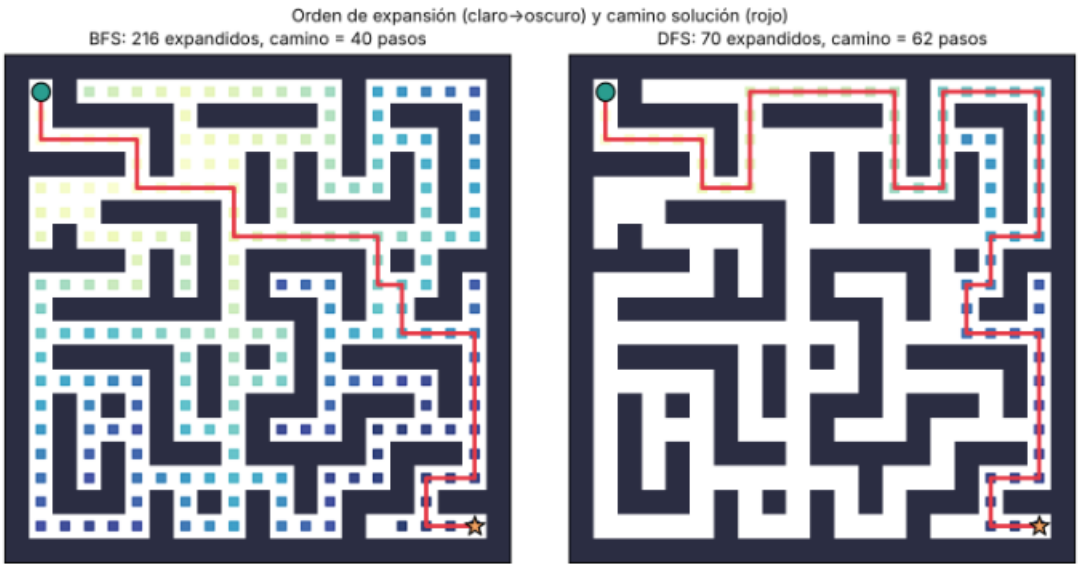
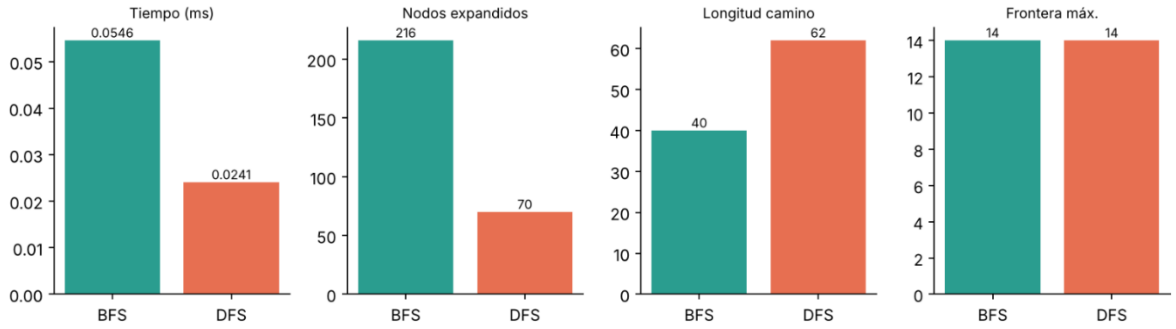


Tabla comparativa

Métrica	BFS	DFS	Observación
Tiempo (mediana de 200 corridas)	0.0546 ms	0.0241 ms	DFS fue 2.3× más rápido en este laberinto
Nodos expandidos	216	70	BFS exploro 99.5% del grafo; DFS 32.3%
Longitud del camino (costo)	40	62	DFS entrega un camino 55% más largo
Frontera maxima (memoria)	14	14	Similar aquí: la frontera de DFS incluye entradas duplicadas
¿Optimo?	Si (= optimo NetworkX: 40)	No	BFS garantiza costo minimo con costo uniforme
¿Completo?	Si	Si (con registro de visitados)	Grafo finito



Complejidad temporal y espacial.

Sea b el factor de ramificacion, d la profundidad de la meta menos profunda y m la profundidad maxima del espacio de estados (UC Berkeley CS 188, s. f.).

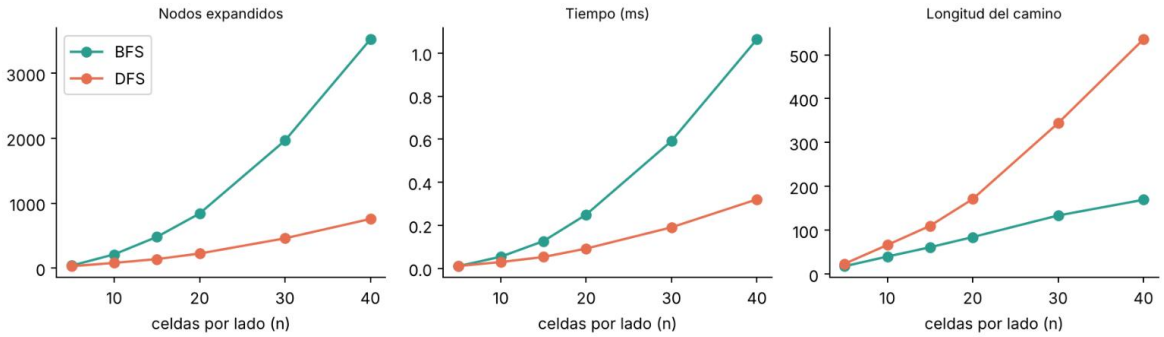
Propiedad	BFS	DFS
Tiempo (árbol)	O(bd)	O(bm)
Tiempo (grafo, con visitados)	O(|V| + |E|)	O(|V| + |E|)

Espacio (árbol)	O(bd): guarda toda la frontera	O(b · m): solo el camino actual y hermanos
Espacio (grafo, con visitados)	O(|V|)	O(|V|) por el conjunto de visitados
Completo	Si (b finito)	Solo con control de ciclos y espacio finito
Optimo	Si, con costo uniforme	No

En este laberinto: b ≤ 3, d = 40 (optima), |V| = 217, |E| = 238. Como se usa búsqueda en grafo con registro de visitados, ambos algoritmos procesan cada nodo y cada arista a lo más una vez, por lo que el costo en el peor caso es O(|V| + |E|) = O(|V|) para los dos (en una cuadricula |E| < 2|V|). La diferencia practica está en cuánto del grafo recorren antes de topar con la meta: BFS debe agotar todos los nodos con distancia menor a d, DFS depende de la suerte del orden de exploración.

Verificación del crecimiento

Se repitió el experimento en laberintos de 5 a 40 celdas por lado (10 semillas por tamaño, promedios). Ambas curvas de nodos expandidos y tiempo crecen de forma aproximadamente lineal con |V|, consistente con O(|V| + |E|); lo que se separa es la calidad del camino.



n(celdas)	5	10	15	20	30	40
BFS expandidos	39	213	482	842	1965	3524
DFS expandidos	30	81	140	225	460	758
BFS longitud	18	40	61	85	134	170
DFS longitud	24	67	110	171	344	532
BFS tiempo (ms)	0.01	0.05	0.13	0.25	0.59	1.06
DFS tiempo (ms)	0.01	0.03	0.05	0.09	0.19	0.32

Comparación

Nodos expandidos y tiempo. En el laberinto base DFS expandio 70 nodos frente a 216 de BFS y tardo 0.024 ms contra 0.055 ms. BFS avanza por ≪ondas≫ de igual distancia y, como la meta está en la esquina más lejana, necesita visitar casi todo el grafo; DFS se ≪compromete≫ con un pasillo y, en este caso, tuvo suerte de llegar sin recorrer todo. En el promedio de laberintos de n = 40, BFS expandio 4.6× más nodos que DFS.
Calidad de la solución. BFS devolvió 40 pasos, igual al optimo calculado independientemente con NetworkX. DFS devolvió 62 pasos; en n = 40 su camino fue en promedio 3.2× más largo que el óptimo. DFS encuentra una ruta, no la más corta: toma desvíos (p. ej. subir y volver a bajar) porque nunca compara alternativas.
Memoria. En este problema las fronteras máximas coinciden (14) porque la cuadricula es angosta (pasillos de 1 celda) y b es pequeño. En espacios con b grande la frontera de BFS crece como b^d mientras que la de DFS crece como $b \cdot m$, que es su ventaja teórica principal.
Caso de borde: laberinto perfecto. Si el laberinto no tiene ciclos (árbol), existe un único camino simple entre S y G; DFS y BFS devuelven la misma ruta y la diferencia se reduce a cuantos nodos visitan. La ventaja en calidad de BFS aparece justamente cuando hay ciclos (rutas alternativas).

Conclusiones

Después de correr los dos algoritmos en el laberinto, queda claro que no hay uno mejor, sino que depende de lo que necesites. BFS me dio el camino más corto: 40 pasos, que es justo el óptimo que calculó NetworkX. El costo fue que tuvo que revisar casi todo el laberinto, 216 de 217 nodos. DFS revisó solo 70 nodos y fue como 2 veces más rápido (0.025 ms contra 0.055 ms). Pero su camino fue de 62 pasos, o sea 55 % más largo, porque se va por el primer pasillo que encuentra y no compara alternativas. Cuando probé con laberintos más grandes (40 celdas por lado), el patrón se mantuvo: BFS revisaba unas 4.6 veces más nodos, pero DFS daba caminos como 3.2 veces más largos.
Entonces, para este laberinto BFS es la mejor opción, porque lo que importa es la ruta más corta y el tamaño es chico, así que la memoria y el tiempo extra no son problema. DFS me serviría más cuando solo quiero saber si existe una salida, cuando hay muchas soluciones posibles o cuando el espacio es enorme y la memoria es limitada, porque guarda menos nodos a la vez. Si los movimientos tuvieran costos distintos (por ejemplo, un camino con lodo que cuesta más), ninguno de los dos sirve, y ahí toca usar Dijkstra o A*, que es lo que sigue en la actividad 2.5.
Algo que me llamó la atención es que, con registro de nodos visitados, los dos tienen la misma complejidad, $O(|V| + |E|)$. La diferencia real no está en qué tan rápido

crecen, sino en qué garantizan: BFS te asegura el camino más corto y DFS solo te asegura que, si hay camino, lo encuentra. Como limitación, solo probé un tipo de laberinto (pasillos angostos y 15 % de ciclos), así que con otros puzles los resultados de memoria podrían cambiar.

Código

```
"""
Actividad 2.4 - Algoritmos de búsqueda ciega (BFS / DFS)

Objetivo:
Resolver un laberinto en cuadricula (movimientos Arriba, Derecha, Abajo,
Izquierda) con Búsqueda en Anchura (BFS) y Búsqueda en Profundidad (DFS),
y comparar: tiempo, nodos expandidos, longitud del camino y memoria
(tamaño máximo de la frontera).

Uso:
python busqueda_ciega.py
Salida:
- Resultados impresos en consola.
- Imágenes en la carpeta ../img/
- Archivo resultados.json en esta misma carpeta.
"""
# ---------------------------- Importaciones ------------------------------
import json                                 # Guardar los resultados en un archivo .json
import random                               # Generar el laberinto de forma pseudoaleatoria
import statistics                           # Calcular la mediana de los tiempos medidos
import time                                 # Medir el tiempo de ejecución (perf_counter)
from collections import deque     # Cola eficiente (FIFO) para BFS
from pathlib import Path             # Manejo de rutas de archivos
import matplotlib
matplotlib.use("Agg")                       # Backend sin ventana: solo guarda imágenes
import matplotlib.pyplot as plt                         # Graficas
import networkx as nx                       # Grafo auxiliar para validar los resultados
from matplotlib.colors import ListedColormap         # Colores pared/libre

# Carpeta donde se guardan las imágenes (../img respecto a este archivo)
IMG = Path(__file__).resolve().parent.parent / "img"
IMG.mkdir(exist_ok=True)                    # Se crea si no existe

# Acciones posibles: (cambio en fila, cambio en columna, nombre).
# El orden es fijo para que los resultados sean reproducibles.
ACCIONES = [(-1, 0, "Arriba"), (0, 1, "Derecha"),
(1, 0, "Abajo"), (0, -1, "Izquierda")]


}# ---------------------------------------------------------------------------
# 1. Generación del laberinto (datos de entrada)
# ---------------------------------------------------------------------------
def generar_laberinto(n_celdas=10, semilla=7, bucles=0.15):
"""Crea un laberinto de (2n+1) x (2n+1). 1 = pared, 0 = celda libre.
Paso 1: laberinto "perfecto" (sin ciclos) con el algoritmo backtracker.
```

```
Paso 2: se abre una fracción `bucles` de paredes internas para crear
ciclos; así hay varios caminos posibles y BFS y DFS pueden
diferir en la calidad de la solución.
Devuelve: (cuadricula, celda_inicial, celda_meta)
"""
rnd = random.Random(semilla)       # Generador con semilla => reproducible
N = 2 * n_celdas + 1               # Tamaño total de la cuadricula
g = [[1] * N for _ in range(N)]    # Se empieza con toda pared
pila = [(1, 1)]                    # Pila del backtracker (celda actual arriba)

vis = {(1, 1)}                     # Celdas ya visitadas
g[1][1] = 0                        # La celda inicial queda libre
while pila:
r, c = pila[-1]                    # Celda en la que estamos
# Vecinos a 2 pasos (saltando la pared intermedia) aun no visitados
vec = [(r + dr * 2, c + dc * 2, dr, dc) for dr, dc, _ in ACCIONES
if 0 < r + dr * 2 < N - 1 and 0 < c + dc * 2 < N - 1
and (r + dr * 2, c + dc * 2) not in vis]
if not vec: # Sin vecinos nuevos: retroceder
pila.pop()
continue
nr, nc, dr, dc = rnd.choice(vec)   # Elegir un vecino al azar
g[r + dr][c + dc] = 0              # Abrir la pared intermedia
g[nr][nc] = 0                      # Abrir la celda vecina
vis.add((nr, nc))                  # Marcarla como visitada
pila.append((nr, nc))              # Avanzar hacia ella

# Paredes internas que separan dos celdas libres: candidatas a abrirse
cand = [(r, c) for r in range(1, N - 1) for c in range(1, N - 1)
if g[r][c] == 1 and ((g[r - 1][c] == 0 and g[r + 1][c] == 0) or
(g[r][c - 1] == 0 and g[r][c + 1] == 0))]
rnd.shuffle(cand)                  # Orden aleatorio reproducible
for r, c in cand[: int(len(cand) * bucles)]:
g[r][c] = 0                        # Se abre la pared => se crea un ciclo

# Inicio en la esquina superior izquierda, meta en la inferior derecha
return g, (1, 1), (N - 2, N - 2)
# ---------------------------------------------------------------------
# 2. Representación del grafo
# ---------------------------------------------------------------------
def construir_grafo(grid):

"""Construye la lista de adyacencia {(fila, col): [vecinos...]}.
Nodo = celda libre.
Arista = dos celdas libres adyacentes (costo 1).
"""
filas, cols = len(grid), len(grid[0])
```

```
ady = {}
for r in range(filas):
for c in range(cols):
if grid[r][c] == 0:                                  # Solo las celdas libres son nodos

# Vecinos: celdas dentro de la cuadricula y libres
ady[(r, c)] = [(r + dr, c + dc) for dr, dc, _ in ACCIONES
if 0 <= r + dr < filas and 0 <= c + dc < cols
and grid[r + dr][c + dc] == 0]
return ady
def a_networkx(ady):

"""Convierte la lista de adyacencia en un grafo de NetworkX (para validar)."""
G = nx.Graph()
for u, vs in ady.items():
for v in vs:
G.add_edge(u, v)                                      # Cada arista se agrega una sola vez
return G

# ----------------------------------------------------------------------
# 3. Algoritmos de búsqueda ciega
# ----------------------------------------------------------------------
def reconstruir (padre, meta):
"""Reconstruye el camino inicio->meta siguiendo los padres desde la meta."""
camino, n = [], meta
while n is not None:                          # El inicio tiene padre None
camino.append(n)
n = padre[n]
return camino[::-1]                          # Se invierte: de inicio a meta
def bfs(ady, inicio, meta):
"""Busqueda en anchura (BFS): la frontera es una COLA (FIFO).
Explora por niveles: primero todos los nodos a distancia 1, luego 2, etc.
Con costo uniforme encuentra el camino más corto (es óptimo).
Devuelve: (camino, orden_de_expansion, tamano_maximo_de_frontera)
"""
frontera = deque([inicio])                              # Cola con el nodo inicial
padre = {inicio: None}                                 # Padres; tambien sirve como "visitados"
orden, max_front = [], 1 # Nodos expandidos y memoria maxima usada
while frontera:
max_front = max(max_front, len(frontera))              # Registrar tamaño de frontera
nodo = frontera.popleft()                               # Sacar el nodo MAS ANTIGUO (FIFO)
orden.append(nodo)                                      # Contarlo como nodo expandido
if nodo == meta:                                       # Prueba de meta


return reconstruir(padre, meta), orden, max_front
for v in ady[nodo]:                                    # Generar sucesores
```

```
if v not in padre:                          # Solo si nunca se había alcanzado
padre[v] = nodo                             # Recordar de donde vino
frontera.append(v)                          # Entra al final de la cola
return None, orden, max_front               # Frontera vacía: no hay solución
def dfs (ady, inicio, meta):

"""Busqueda en profundidad (DFS): la frontera es una PILA (LIFO).
Avanza por una rama hasta que no puede continuar y entonces retrocede.
Es más barata en memoria pero NO garantiza el camino más corto.
Devuelve: (camino, orden_de_expansion, tamano_maximo_de_frontera)
"""
frontera = [inicio]                         # Pila (lista de Python) con el inicio
padre = {inicio: None}                      # Padres para reconstruir el camino
visitado, orden, max_front = set (), [], 1
while frontera:
max_front = max(max_front, len(frontera))   # Registrar tamaño de frontera
nodo = frontera.pop()                       # Sacar el nodo MAS RECIENTE (LIFO)
if nodo in visitado:                        # Puede haber duplicados en la pila
continue
visitado.add(nodo)                          # Marcar como expandido
orden.append(nodo)                          # Contarlo como nodo expandido
if nodo == meta:                            # Prueba de meta
return reconstruir(padre, meta), orden, max_front

# Se apilan en reversa para que "Arriba" sea lo primero en explorarse
for v in reversed(ady[nodo]):
if v not in visitado:
padre[v] = nodo                             # Padre más reciente (el que usara la pila)
frontera.append(v)
return None, orden, max_front               # No hay solución
def acciones_de(camino):

"""Traduce un camino de celdas a la lista de acciones (Arriba, Derecha...)."""
nombres = {(dr, dc): n for dr, dc, n in ACCIONES}
# Para cada par de celdas consecutivas se busca que movimiento las une
return [nombres[(b[0] - a[0], b[1] - a[1])] for a, b in zip(camino, camino[1:])]
def medir(algoritmo, ady, ini, meta, repeticiones=200):

"""Ejecuta un algoritmo varias veces y devuelve sus métricas.
El tiempo reportado es la MEDIANA de las repeticiones (en milisegundos),
lo que reduce el ruido causado por el sistema operativo.
"""
tiempos = []
for _ in range(repeticiones):
t0 = time.perf_counter()                    # Reloj de alta resolución
res = algoritmo(ady, ini, meta)             # Se ejecuta la búsqueda
tiempos.append((time.perf_counter() - t0) * 1000) # s -> ms
```

```
camino, orden, mf = res                    # Resultado de la última ejecución
return {"camino": camino, "orden": orden,
"tiempo_ms": statistics.median(tiempos),
"expandidos": len(orden),                  # Nodos expandidos
"longitud": len(camino) - 1 if camino else None,          # Pasos del camino
"max_frontera": mf} # Memoria maxima

# ----------------------------------------------------------------------
# 4. Graficas
# ----------------------------------------------------------------------
CMAP = ListedColormap(["#ffffff", "#2b2d42"]) # 0 = blanco (libre), 1 = oscuro
(pared)
def dibujar(grid, ini, meta, ax, camino=None, orden=None, titulo=""):

"""Dibuja el laberinto; opcionalmente el orden de expansión y el camino."""
ax.imshow(grid, cmap=CMAP, interpolation="nearest")              # Paredes y
pasillos
if orden: # Nodos expandidos
ys, xs = zip(*orden)

# El color va de claro a oscuro según el orden en que se expandieron
ax.scatter(xs, ys, c=range(len(orden)), cmap="YlGnBu", s=26,
marker="s", alpha=0.85)
if camino:                  # Camino solución
ys, xs = zip(*camino)
ax.plot(xs, ys, color="#e63946", lw=2.4)
ax.scatter(ini[1], ini[0], c="#2a9d8f", s=110, marker="o", zorder=5, edgecolors="k")
# Inicio
ax.scatter(meta[1], meta[0], c="#f4a261", s=140, marker="*", zorder=5,
edgecolors="k") # Meta
ax.set_title(titulo, fontsize=10)
ax.set_xticks([]); ax.set_yticks([])                # Sin ejes numéricos
def figuras(grid, ini, meta, rb, rd):

"""Genera y guarda las figuras del reporte en ../img/."""


# Figura 1: laberinto original
fig, ax = plt.subplots(figsize=(4.6, 4.6))
dibujar(grid, ini, meta, ax,
título=f"Laberinto {len(grid)}x{len(grid[0])} (S=verde, G=estrella)")
fig.tight_layout(); fig.savefig(IMG / "laberinto.png", dpi=170); plt.close(fig)

# Figura 3: soluciones y orden de exploración de BFS y DFS lado a lado
fig, axs = plt.subplots(1, 2, figsize=(9.2, 4.7))
for ax, r, n in zip(axs, (rb, rd), ("BFS", "DFS")):
dibujar(grid, ini, meta, ax, r["camino"], r["orden"],
```

```
f"{n}: {r['expandidos']} expandidos, camino = {r['longitud']} pasos")
fig.suptitle("Orden de expansión (claro->oscuro) y camino solucion (rojo)",
fontsize=10)
fig.tight_layout(); fig.savefig(IMG / "soluciones.png", dpi=170); plt.close(fig)
# Figura 4: barras comparando las cuatro métricas
fig, axs = plt.subplots(1, 4, figsize=(10.5, 3.1))
claves = [("tiempo_ms", "Tiempo (ms)"), ("expandidos", "Nodos expandidos"),
("longitud", "Longitud camino"), ("max_frontera", "Frontera max.")]
for ax, (k, t) in zip(axs, claves):
b = ax.bar(["BFS", "DFS"], [rb[k], rd[k]], color=["#2a9d8f", "#e76f51"])
ax.bar_label(b, fmt="%.3g", fontsize=8) # Valor sobre cada barra
ax.set_title(t, fontsize=9); ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout(); fig.savefig(IMG / "comparativa.png", dpi=170); plt.close(fig)

# Figura 2: ejemplo pequeño que muestra cuadricula -> grafo
peq = [[0, 0, 0, 1], [1, 1, 0, 1], [0, 0, 0, 0], [0, 1, 1, 0]]
G = a_networkx(construir_grafo(peq))
pos = {n: (n[1], -n[0]) for n in G}                    # Posición = (col, -fila)
fig, axs = plt.subplots(1, 2, figsize=(7.4, 2.9))
axs[0].imshow(peq, cmap=CMAP); axs[0].set_title("Cuadricula (negro = pared)",
fontsize=9)
axs[0].set_xticks([]); axs[0].set_yticks([])
nx.draw(G, pos, ax=axs[1], with_labels=True, node_size=520, font_size=6,
node_color="#a8dadc", edge_color="#457b9d")
axs[1].set_title("Grafo G=(V,E) equivalente", fontsize=9)
fig.tight_layout(); fig.savefig(IMG / "grafo_ejemplo.png", dpi=170); plt.close(fig)
def escalamiento():

"""Compara BFS y DFS en laberintos de tamaños crecientes.
Para cada tamaño se promedian 10 laberintos distintos (semillas 0..9).
Sirve para comprobar empíricamente el crecimiento ~ O(|V| + |E|).
"""
tams = [5, 10, 15, 20, 30, 40]                         # Celdas por lado
datos = {"BFS": {"exp": [], "t": [], "lon": []},
"DFS": {"exp": [], "t": [], "lon": []}}
for n in tams:
acum = {a: {"exp": [], "t": [], "lon": []} for a in datos}
for s in range(10):                                    # 10 semillas por tamaño
g, i, m = generar_laberinto(n, semilla=s, bucles=0.15)
ady = construir_grafo(g)
for nombre, f in (("BFS", bfs), ("DFS", dfs)):
r = medir(f, ady, i, m, repeticiones=15)
acum[nombre]["exp"].append(r["expandidos"])
acum[nombre]["t"].append(r["tiempo_ms"])
acum[nombre]["lon"].append(r["longitud"])
for a in datos:                                        # Promedio por tamaño
for k in ("exp", "t", "lon"):
```

```
datos[a][k].append(statistics.mean(acum[a][k]))

# Figura 5: tres graficas de líneas (expandidos, tiempo y longitud)
fig, axs = plt.subplots(1, 3, figsize=(10.5, 3.2))
for ax, k, t in zip(axs, ("exp", "t", "lon"),
("Nodos expandidos", "Tiempo (ms)", "Longitud del camino")):
for a, col in (("BFS", "#2a9d8f"), ("DFS", "#e76f51")):
ax.plot(tams, datos[a][k], marker="o", label=a, color=col)
ax.set_title(t, fontsize=9); ax.set_xlabel("celdas por lado (n)")
ax.spines[["top", "right"]].set_visible(False)
axs[0].legend()
fig.tight_layout(); fig.savefig(IMG / "escalamiento.png", dpi=170); plt.close(fig)
return {"tamanos": tams, **datos}
# ----------------------------------------------------------------------
# 5. Programa principal
# ----------------------------------------------------------------------
def main():
# a) Datos de entrada: laberinto, grafo y sus tamanos
grid, ini, meta = generar_laberinto (n_celdas=10, semilla=7, bucles=0.15)
ady = construir_grafo(grid)
V = len(ady) # Numero de nodos
E = sum(len(v) for v in ady.values()) // 2 # Aristas (cada una cuenta 2 veces)
G = a_networkx(ady)

# Validación: nuestro grafo coincide con el de NetworkX
assert G.number_of_nodes() == V and G.number_of_edges() == E
print(f"Laberinto {len(grid)}x{len(grid[0])} | V={V} | E={E} | ciclos={E - V + 1}")



print(f"Inicio={ini} Meta={meta} conexo={nx.is_connected(G)}")

# b) Ejecutar y medir ambos algoritmos
rb = medir(bfs, ady, ini, meta)
rd = medir(dfs, ady, ini, meta)

# c) Validación: BFS debe igualar el camino optimo calculado por NetworkX
optimo = nx.shortest_path_length(G, ini, meta)
assert rb["longitud"] == optimo, "BFS debe ser optimo"

# d) Mostrar resultados en consola
for n, r in (("BFS", rb), ("DFS", rd)):
print(f"{n}: pasos={r['longitud']} expandidos={r['expandidos']} "
f"frontera_max={r['max_frontera']} tiempo={r['tiempo_ms']:.4f} ms")
print("Camino BFS (acciones):", ", ".join(acciones_de(rb["camino"])))
print("Camino DFS (acciones):", ", ".join(acciones_de(rd["camino"])))
```

```
# e) Graficas y experimento de escalamiento
figuras(grid, ini, meta, rb, rd)
esc = escalamiento()

# f) Guardar todo en resultados.json (el campo "orden" se omite por tamaño)
out = {"V": V, "E": E, "ciclos": E - V + 1, "inicio": ini, "meta": meta,
"optimo_networkx": optimo, "grid": grid,
"BFS": {k: v for k, v in rb.items() if k != "orden"},
"DFS": {k: v for k, v in rd.items() if k != "orden"},
"acciones_BFS": acciones_de(rb["camino"]),
"acciones_DFS": acciones_de(rd["camino"]),
"escalamiento": esc}
Path(__file__).with_name("resultados.json").write_text(json.dumps(out, indent=1))
print("Listo: imágenes en", IMG)
# Se ejecuta main() solo si el archivo se corre directamente
if __name__ == "__main__":
main()
```

Referencias

NetworkX developers. (s. f.). *NetworkX documentation*. https://networkx.org
Python Software Foundation. (s. f.). *collections — Container datatypes*. En *The Python standard library*. https://docs.python.org/3/library/collections.html

UC Berkeley CS 188. (s. f.). *1.3 Uninformed search*. En *Introduction to artificial intelligence*. https://inst.eecs.berkeley.edu/~cs188/textbook/search/uninformed.html

IA que utilicé: Claude (Anthropic), modelo Sonnet 5.5 [Modelo de lenguaje de gran tamaño]

Prompt: Realiza la actividad 2.4 Algoritmos de búsqueda ciega (BFS/DFS). Código + comparativa. Implementar BFS y DFS en Python para resolver un laberinto o puzle. Cumplir cada punto de las instrucciones generales y la lista de cotejo.

Uso: Generación del código, graficas

Verificación: EL código se ejecutó para ver si cumplía con lo solicitado, que los BFS coincidieran con el camino optimo de NetworkX y que |V|, |E| coincide con el grafo NetworkX.

IA que utilice: Gemini (Google, 2026).

Prompt: Realiza la documentación de la información mas importante que se encuentra en los códigos y que abarque lo solicitado de la lista de cotejo.

Uso: Que el documento si cumpla con todos los puntos y de la lista de cotejo.

Verificación: Que la información coincida con lo que esta generada en el código.